

Filière: Systèmes Intelligents, Communicants et Mobiles
SICoM

MACHINE D'ETAT FINIS

FSM : **F**INITE **S**TATE **M**ACHINE

&

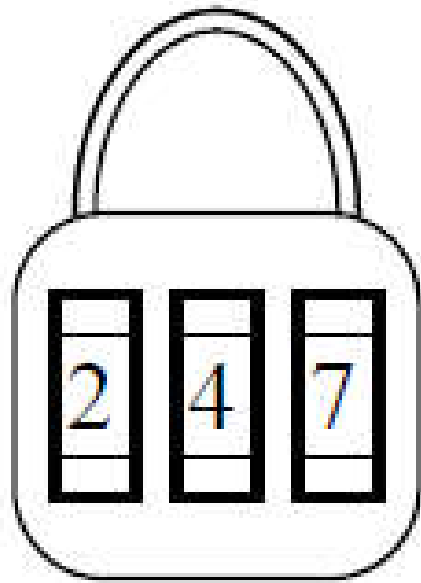
VHDL

*Année universitaire
2025/2026*

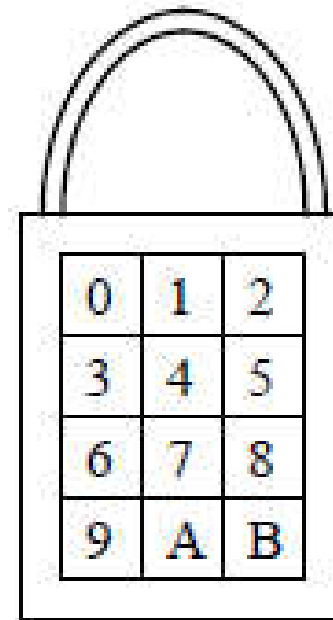
Plan

- ❖ Introduction
- ❖ Structure d'une FSM
- ❖ Représentation d'une FSM
- ❖ Machines de Moore et de Mealy
- ❖ Description VHDL des FSM

Introduction



serrure mécanique

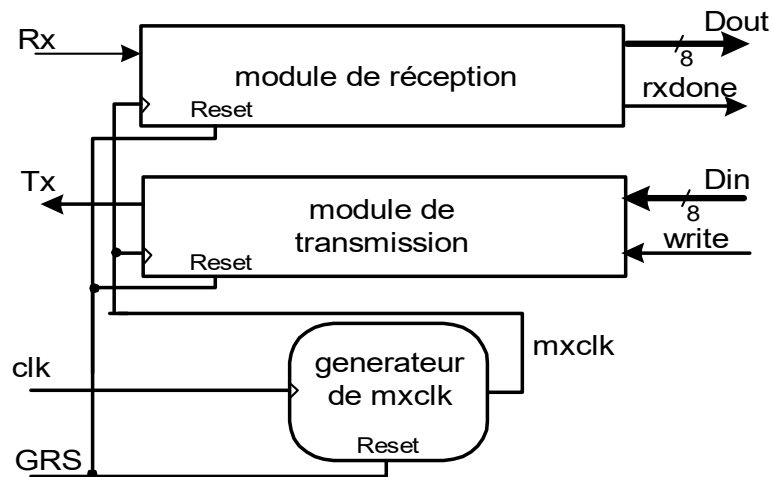
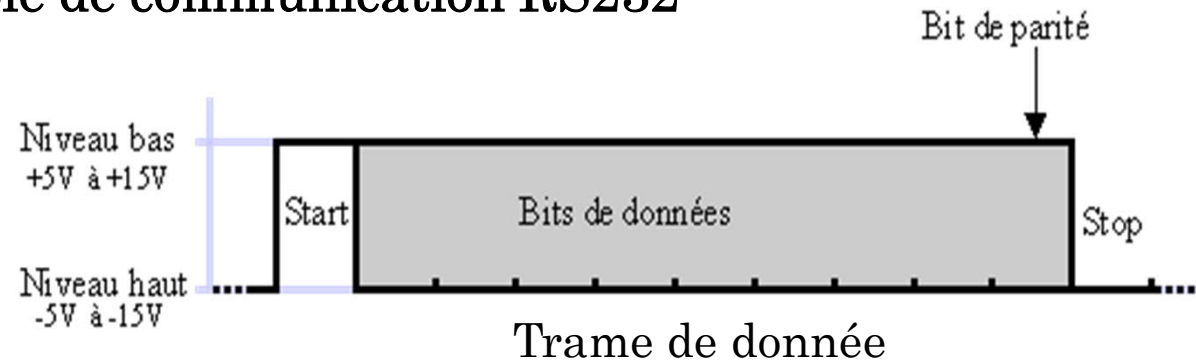


serrure à digicode

Introduction

Exemple: Implémentation d'un UART sur FPGA

Protocole de communication RS232



Architecture de l'UART

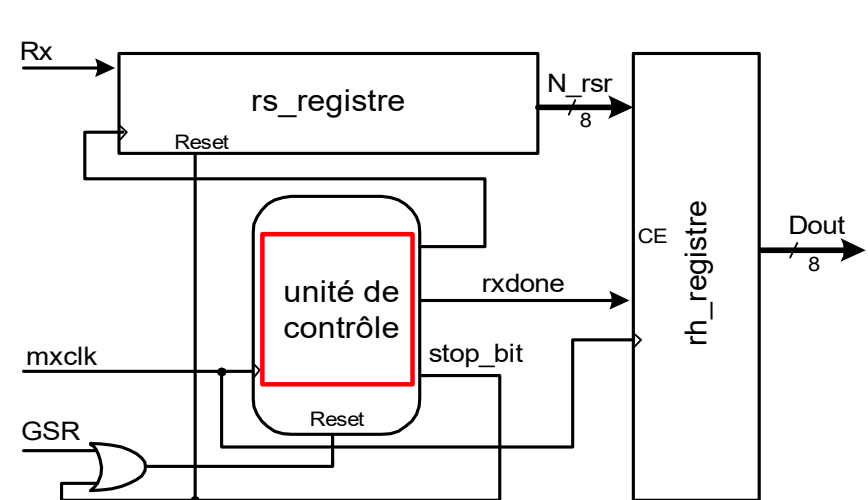
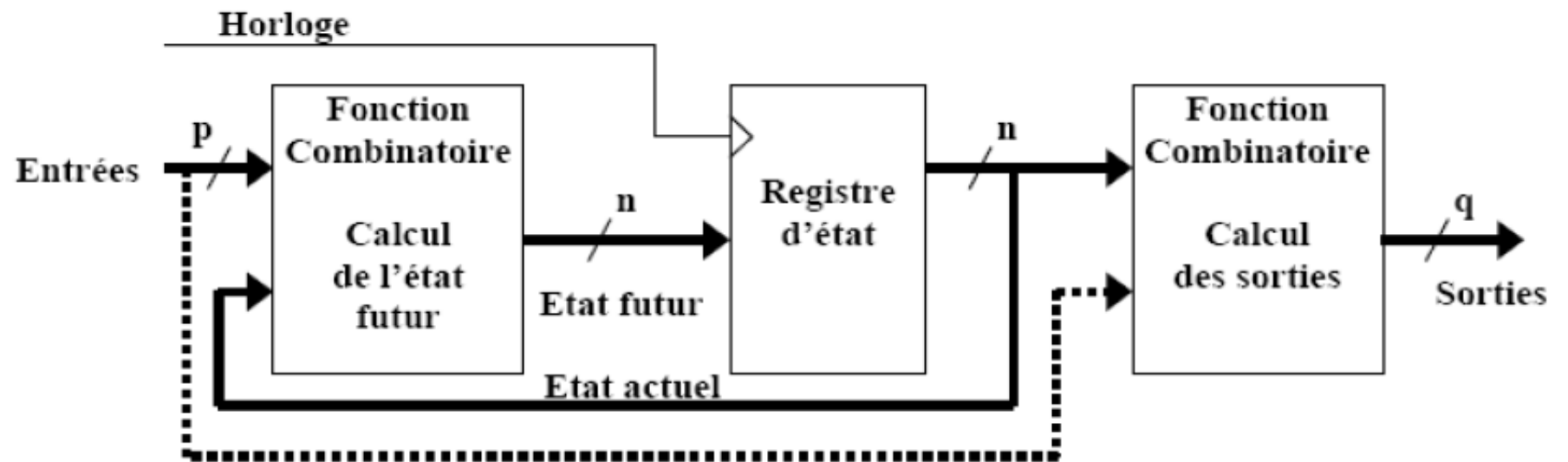


Schéma bloc du module de réception

Introduction

- Une Machine à états finis (**FSM**) est un modèle mathématique de calcul utilisé dans de nombreux domaines (conception de programmes informatiques, protocoles de communication, contrôle des processus, analyse linguistique, ...).
- Une **FSM** est un circuit **séquentiel** dont les sorties dépendent d'un état et éventuellement des entrées.
- Toute fonction séquentielle est caractérisée par un effet de **mémoire** : capacité à retenir les événements antérieurs.
- Une **FSM** est généralement utilisé comme circuit de **contrôle** qui permet de gérer les échanges des données entre les différentes unités d'un système.

Structure d'une FSM



Structure d'une machine à états

Structure d'une FSM

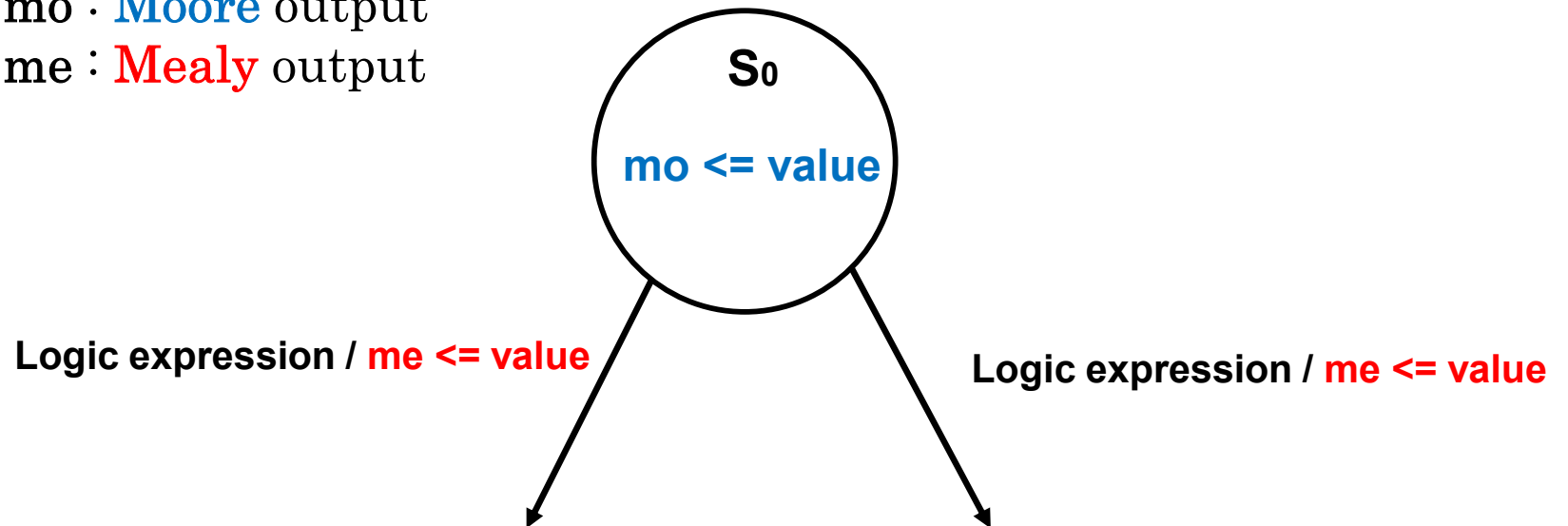
- ❑ Un bloc combinatoire qui calcule l'état futur en fonction de l'état présent et des entrées du système.
- ❑ Un registre d'état cadencé par l'horloge du système, met à jour l'état du système. Il mémorise l'état actuel du système en l'actualisant à des instants précis.
- ❑ Un bloc combinatoire qui calcule les sorties du système en fonction de :
 - ✓ L'état actuel. On obtient une **machine de Moore**.
 - ✓ L'état présent et des entrées . On a une **machine de Mealy**.

Représentation d'une FSM

- Une FSM est décrite par un diagramme à état.
- Chaque état contient un certain nombre d'informations: état, sorties, entrées

mo : **Moore** output

me : **Mealy** output



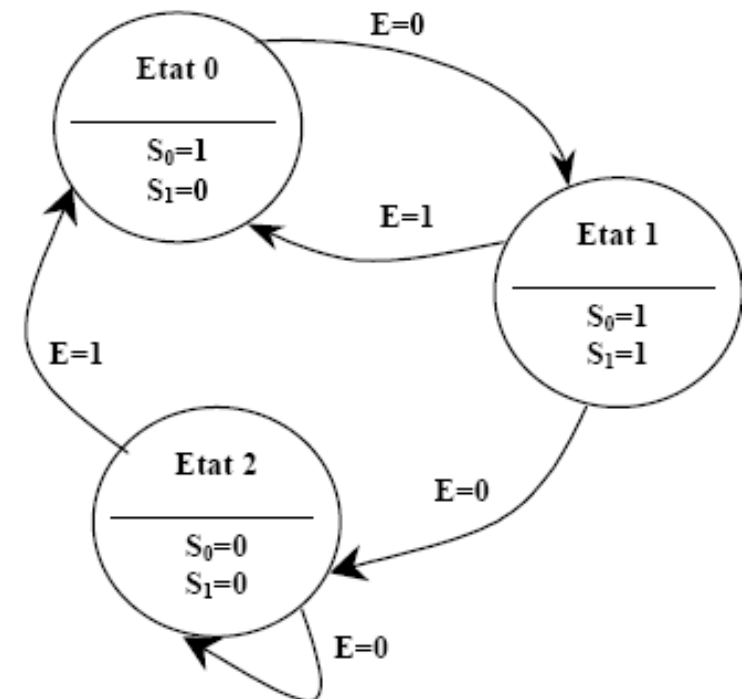
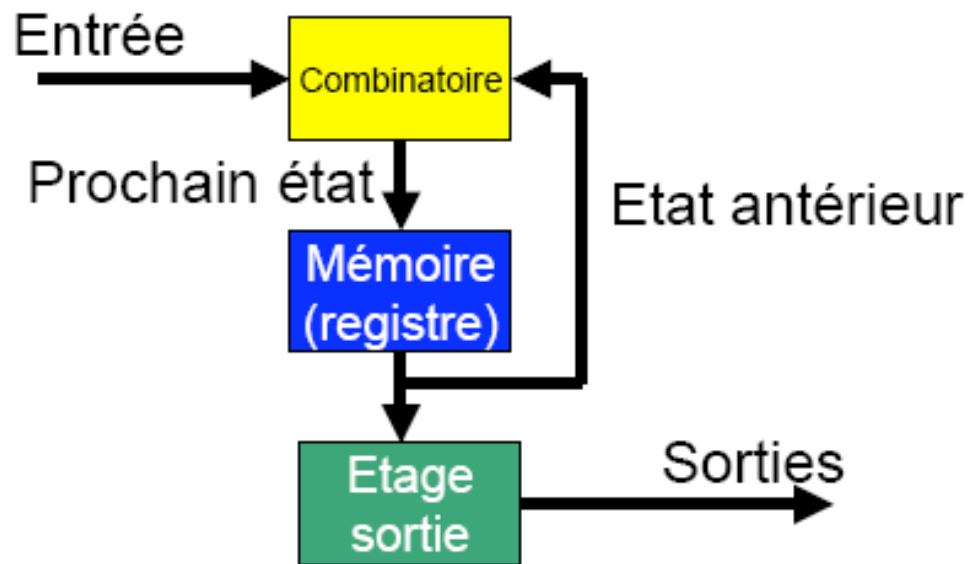
Notation for a state

Représentation d'une FSM

- On associe à chaque valeur possible du registre d'état, une case sous forme d'un cercle.
- L'évolution du système est représentée par des flèches représentant les transitions.
- Pour qu'une transition soit activée il faut que les trois conditions suivantes soient vérifiées :
 - ✓ Le système se trouve dans l'état «source » considéré.
 - ✓ La condition de réalisation sur les entrées est vraie.
 - ✓ Un front actif de l'horloge survient.

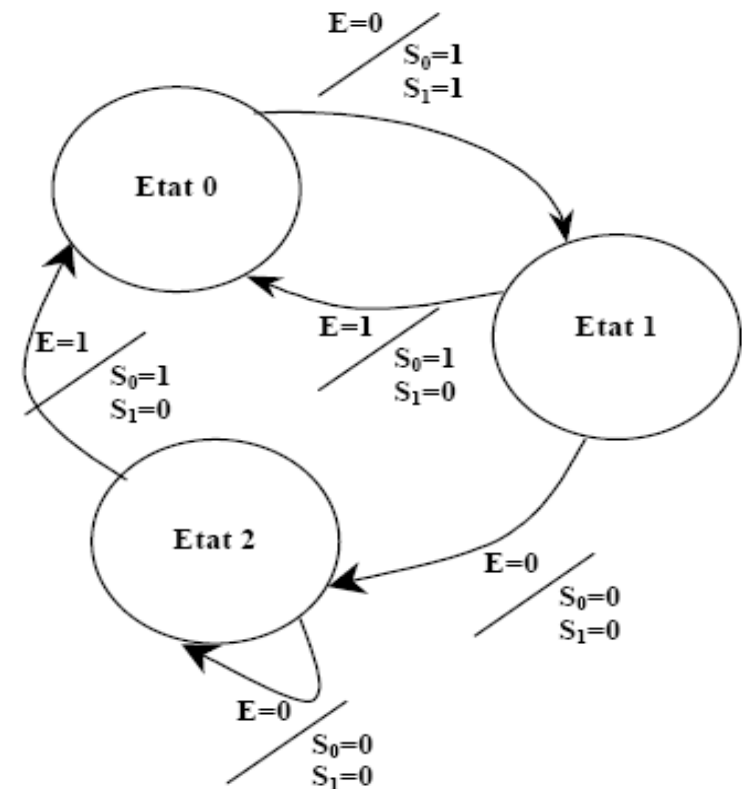
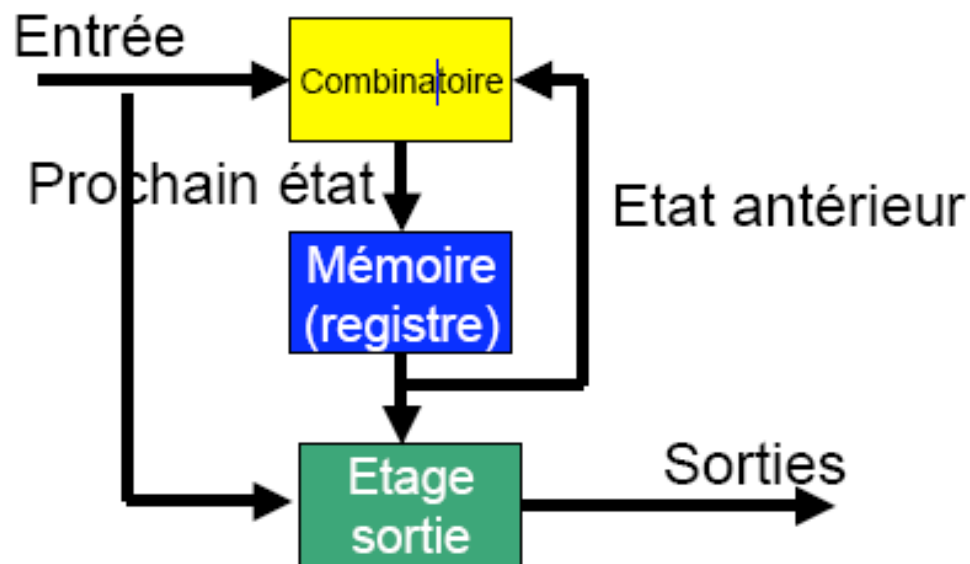
Machine de Moore

- Pour les machines de **Moore**, les sorties évoluent après l'activation de la transition.
- les sorties changent de manière **synchrone** sur un front d'horloge.



Machine de Mealy

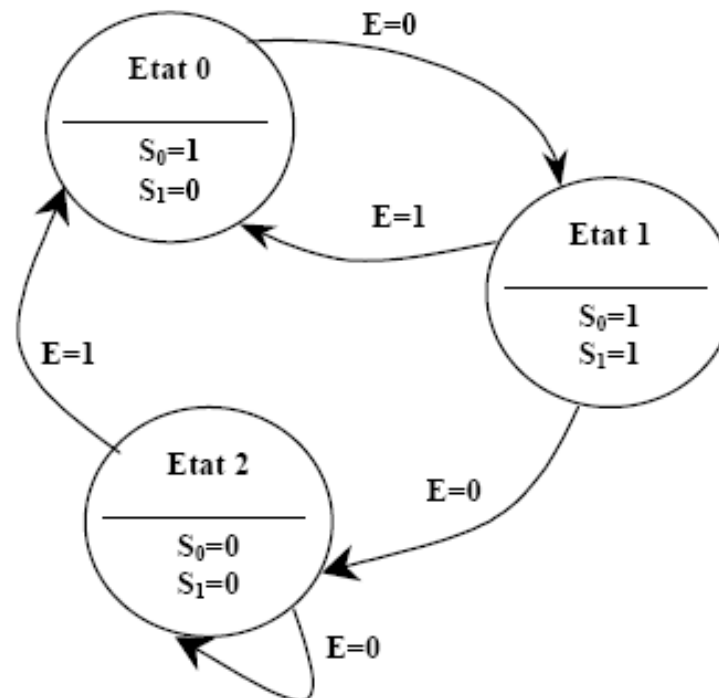
- Pour les machines de **Mealy** les sorties évoluent après l'évolution des entrées.
- Une machine de Mealy est donc **asynchrone**.
- prise en compte immédiate d'un changement en entrée.
- un temps de réponse réduit d'un cycle par rapport à la machine de Moore



Description VHDL des FSM

Exemple:

La description du système se fait par un nombre fini d'états. Ci-dessous la représentation schématique d'un système à 3 états, une entrée E, une entrée d'horloge, une autre de remise à zéro et 2 sorties (S0 et S1) :



Description VHDL des FSM

```
library ieee;  
use ieee.std_logic_1164.all;  
  
ENTITY FSM IS  
PORT ( clk,rst,E   :IN STD_LOGIC;  
        S0, S1      :OUT STD_LOGIC  
        );  
END FSM;
```

Description VHDL des FSM

```
ARCHITECTURE diagramme OF FSM IS
  TYPE etat_3 IS (Etat0,Etat1,Etat2);
  SIGNAL etat,etat_suiv :etat_3 := Etat0;

BEGIN
  --Process synchrone:
  PROCESS (clk,rst)
    BEGIN
      if rst = '1' THEN etat <= Etat0;
      elsif rising_edge(clk) THEN
        etat <= etat_suiv ;
      end if;
    END PROCESS;
```

Description VHDL des FSM

--LE PROCESS ASYNCHRONE :

```
process (etat, E)
  BEGIN
    CASE etat IS
      WHEN Etat0 => if E= '0' then etat_suiv <= Etat1;
                     else etat_suiv <= Etat0;
                     end if;
      WHEN Etat1 => if E= '0' then etat_suiv <= Etat2;
                     else etat_suiv <= Etat0;
                     end if;
      WHEN Etat2 => if E= '1' then etat_suiv <= Etat0;
                     else etat_suiv <= Etat2;
                     end if;
      WHEN others => etat_suiv <= Etat0;
    END CASE;
  end process ;
```

Description VHDL des FSM

```
----- Moore output logic-----  
---  
process (etat)  
  BEGIN  
    CASE etat IS  
      WHEN Etat0 => S0 <= '1'; S1<= '0';  
      WHEN Etat1 => S0 <= '1'; S1<= '1';  
      WHEN Etat2 => S0 <= '0'; S1<= '0';  
    END CASE;  
  END process ;  
end diagramme ;
```


Description VHDL des FSM

Exemple de simulation:

TESTBENCH:

```
LIBRARY ieee ;  
LIBRARY work;  
USE ieee.std_logic_1164.all ;  
Use work.all;  
  
ENTITY FSM_tb IS  
END ;  
  
ARCHITECTURE arch_tb OF FSM_tb IS  
    SIGNAL clk, rst, E, S0, S1 : std_logic ;
```

Description VHDL des FSM

```
COMPONENT FSM
    PORT ( clk, rst, E   : in std_logic ;
          S0, S1       : out std_logic);
END COMPONENT ;

BEGIN
    DUT   : FSM PORT MAP (clk, rst, E, S0, S1);

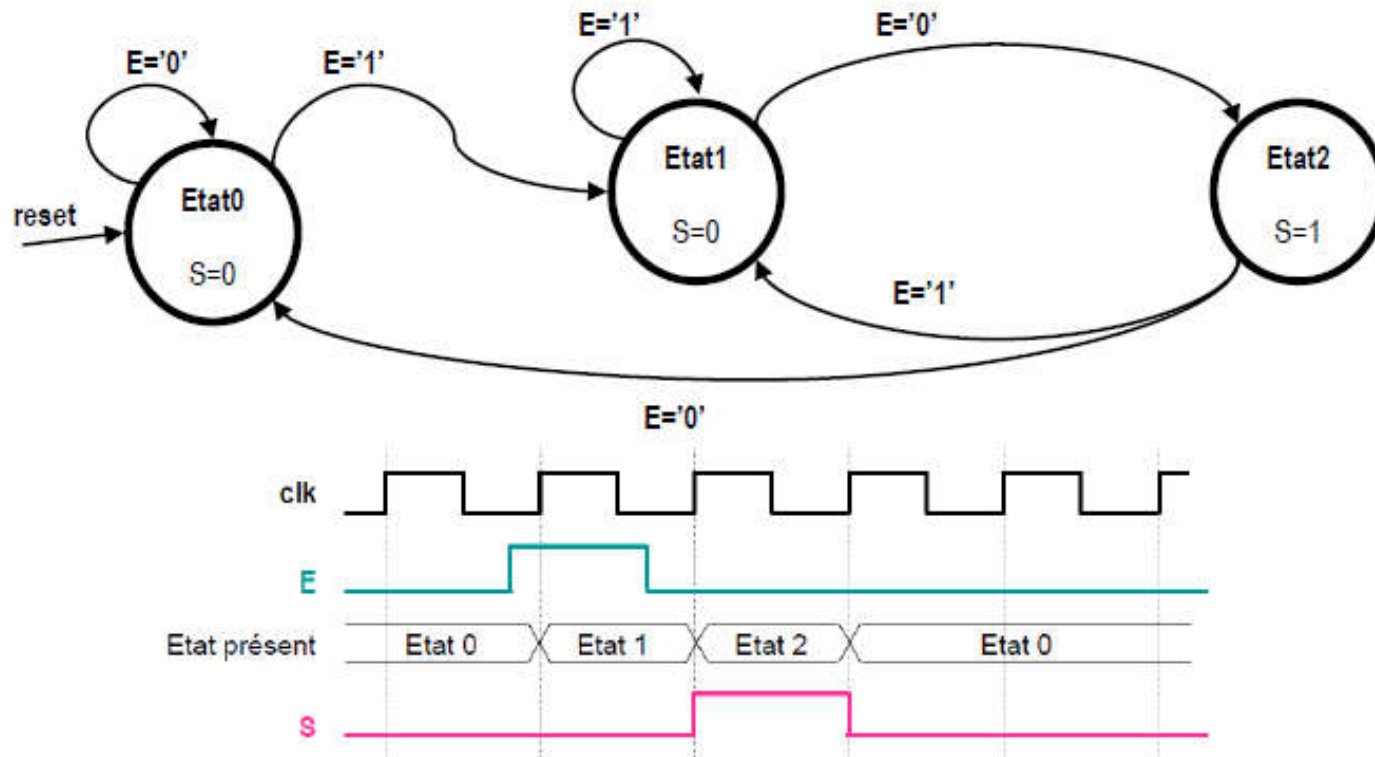
    clk_p: process
    begin
        horl<='0'; wait for 20 ns;
        horl<='1'; wait for 20 ns;
    end process clk_p;

    rst<='1' , '0' after 10 ns;
    E<='1', '0' after 100 ns, '1' after 160 ns;

END arch_tb;
```

Exemple: Détecteur de séquence

La machine de Moore reconnaissant la séquence 10 est la suivante:



Exercice d'application

On désire réaliser une fonction dont la sortie S recopie l'état logique présent sur son entrée E si celle-ci est restée stable après 2 coups d'horloge successifs.

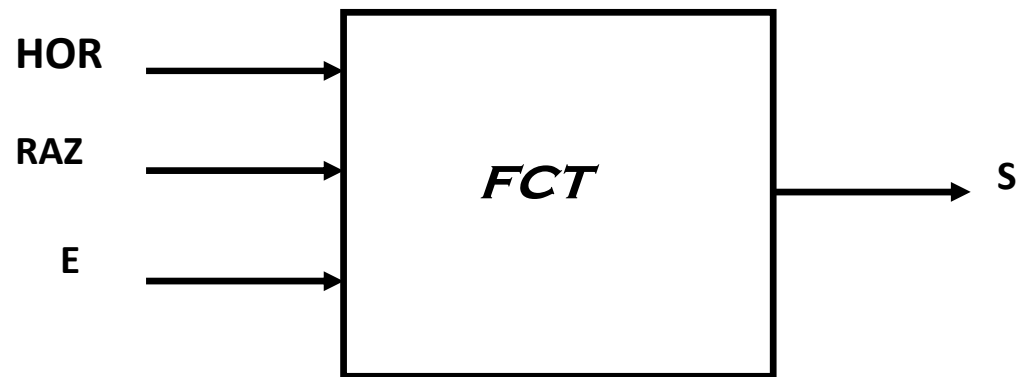
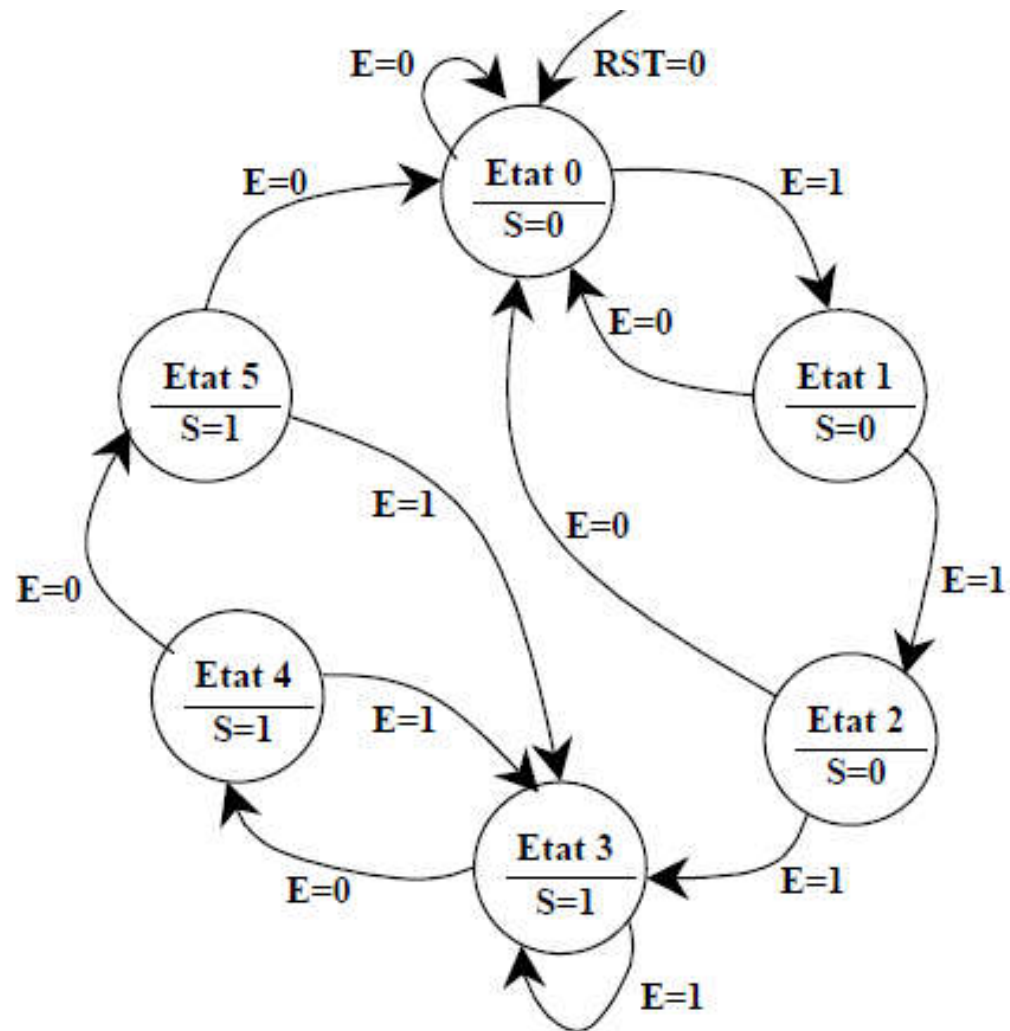


Diagramme à état:



Description VHDL: utilisation d'un seul process

```
library ieee ;
use ieee.std_logic_1164.ALL;

entity MAE is      --MAE = Machine A Etat
  port ( E,RST,HOR : in  STD_LOGIC ;
        S          : out STD_LOGIC );
end MAE;

architecture COMPORTEMENT of MAE is
  signal REG_ETAT : STD_LOGIC_VECTOR(2 downto 0);
  process (HOR,RST)
  begin
    if RST='0' then REG_ETAT <= "000";
    elsif (HOR'event and HOR='1') then
      case REG_ETAT is
        when "000" => S = '0' ;
                      if E = '1' then REG_ETAT <= "001";
                      else REG_ETAT <= "000";
                      end if ;
        when "001" => S = '0' ;
                      if E = '1' then REG_ETAT <= "010";
                      else REG_ETAT <= "000";
                      end if ;
        when "010" => S = '0' ;
                      if E = '1' then REG_ETAT <= "011";
                      else REG_ETAT <= "000";
                      end if ;
        when "011" => S = '1' ;
                      if E = '1' then REG_ETAT <= "011";
                      else REG_ETAT <= "100";
                      end if ;
        when "100" => S = '1' ;
                      if E = '1' then REG_ETAT <= "011";
                      else REG_ETAT <= "101";
                      end if ;
        when "101" => S = '0' ;
                      if E = '1' then REG_ETAT <= "011";
                      else REG_ETAT <= "000";
                      end if ;
        when others => REG_ETAT <= "000";
      end case ;
    end if ;
  end process ;
end COMPORTEMENT ;
```